

1. Production is Down, No Logs Available. Your app is down. The logging system is also down. No logs. What do you do?

Ans :

With production down and no logs available, my priority is to restore service quickly rather than spend time debugging blindly.

First, I'd check recent changes like deployments, config updates, or infrastructure changes to see if the issue correlates.

If a recent deployment is suspected, I'd initiate a rollback to the last stable version, ensuring it's safe with respect to database changes.

In parallel, I'd use available signals like health checks, metrics (CPU, memory, error rates), and system status to narrow down whether the issue is at the application, infrastructure, or network level.

If rollback isn't feasible, I'd try to isolate the failure by restarting only affected components or shifting traffic to healthy instances if available.

Once the system is stable, I'd investigate both the root cause of the failure and why logging was unavailable, since that's a critical gap in observability.

2. Deployment Succeeded but Users See Old Version. CI/CD pipeline says success, pods are running fine. But users still see the old UI. What's your move?

Ans:

Since the deployment succeeded and pods are healthy, I wouldn't assume an application failure. This usually points to caching or traffic routing issues.

First, I'd verify that the new version is actually deployed by checking the image tag of running pods using `kubectl get pods` or `kubectl describe pod`.

If the correct version is running, I'd check for caching issues by testing via incognito or curl and reviewing cache headers or CDN behavior.

If caching is not the issue, I'd verify service selectors and ensure traffic is routed to the correct pods, and then check ingress or load balancer configuration for any stale backends or misrouting.

Based on findings, I'd either invalidate cache, fix deployment/image tagging, or correct routing configuration rather than redeploying blindly.

3. Sudden Traffic Spike. Traffic increased 10x unexpectedly. The app is slowing down. What do you do FIRST?

Ans: In a sudden 10x traffic spike with slowdown, I'd first verify whether the system is actually resource-bound instead of scaling blindly.

I'd check real-time metrics like CPU, memory, latency, and error rates using tools like `kubectl top` and monitoring dashboards.

If the application layer is saturated, I'd scale horizontally by increasing replicas or triggering autoscaling.

At the same time, I'd check for downstream bottlenecks such as database limits, slow queries, or external dependencies, since scaling the app alone won't help if they are the constraint.

I'd also apply rate limiting or traffic control if needed to prevent system overload.

Finally, I'd validate whether the traffic spike is expected or abnormal, as that influences further action.

4. DB is Slow, CPU is Low. Database queries are slow, but CPU usage is normal.

What's your approach?

Ans: If database queries are slow but CPU usage is low, I wouldn't consider this a compute problem. It usually points to inefficient queries, indexing issues, or I/O bottlenecks.

First, I'd check slow query logs to identify which queries are taking time. Then I'd analyze execution plans to see if indexes are missing or not being used properly.

I'd also look at I/O metrics like disk read/write latency, since slow storage can cause delays even when the CPU is idle.

Next, I'd check for locking or blocking issues, where one query is holding up others.

If connection limits are high, I'd verify connection pooling and whether too many idle or long-running connections are affecting performance.

Based on findings, I'd optimize queries, add appropriate indexes, or tune database configuration rather than scaling CPU blindly.

5. Private Repo Leaked Publicly for 10 Minutes. Someone accidentally made the repo public. What's FIRST priority?

Ans:

If a private repository was exposed publicly, my first priority is to assume all secrets are compromised and immediately rotate them.

That includes API keys, access tokens, database credentials, and any sensitive configuration present in the repo.

In parallel, I'd restrict access by making the repository private again and auditing logs to understand if any unauthorized access occurred during that window.

After containment, I'd remove secrets from the codebase, move them to a secure solution like a secrets manager, and enforce practices to prevent this in the future, such as secret scanning and access controls.

6. Pod is Running but App Not Accessible. Kubernetes pod status = Running

But app is not reachable. What do you do now?

Ans: If the pod is in Running state but the application is not accessible, I wouldn't assume the app is healthy. Running only means the container started, not that it's serving traffic.

First, I'd verify whether the application inside the pod is actually listening on the expected port by checking container logs and, if needed, exec into the pod to validate the process.

Next, I'd check the Kubernetes Service configuration—especially the selector and targetPort—to ensure it correctly maps to the pod labels and container port.

If the service looks correct, I'd move up the traffic path and verify Ingress or Load Balancer configuration to ensure routing is properly set.

I'd also check readiness probes, because if the pod isn't ready, it won't receive traffic even though it's running.

Based on where the break occurs—application, service, or ingress—I'd fix the configuration rather than restarting components blindly.

7. Disk is Full on One Node Only. the cluster has multiple nodes, but one node is full.

Ans: If the disk is full on only one node in a cluster, I'd first prevent impact by isolating that node rather than touching files blindly.

I'd cordon and drain the node to stop new pods from scheduling and safely move existing workloads to other nodes.

Then I'd investigate what's consuming disk space—checking container logs, unused images, or orphaned volumes using tools like `du`, `df`, and container runtime commands.

If the issue is due to log growth, I'd clean up or rotate logs; if it's unused images or containers, I'd prune them safely.

After resolving the root cause, I'd bring the node back into the cluster and also implement preventive measures like log rotation, disk monitoring, and alerting to avoid recurrence.

8. Pipeline Takes 40 Minutes Suddenly Earlier it took 10 mins. No major code change.

What do you check?

Ans: If a pipeline suddenly increases from 10 minutes to 40 without major code changes, I'd first compare recent runs to identify which stage is taking longer instead of treating the pipeline as a black box.

I'd check stage-level logs and timing to pinpoint the slowdown—whether it's build, test, or deployment.

Then I'd correlate with recent changes outside code, such as dependency updates, base image changes, or infrastructure issues like slower runners or network latency.

I'd also check for caching issues—like cache misses or invalidation—since loss of cache can significantly increase build time.

If the issue is resource-related, I'd verify runner capacity (CPU/memory contention) and whether jobs are queued or throttled.

Based on findings, I'd restore caching, fix dependency or image changes, or optimize the slow stage rather than simply increasing runners blindly.

9. App Works Locally but Fails in Production. Same Docker image. What's your suspicion?

Ans: If the same Docker image works locally but fails in production, my first suspicion is an environment or configuration difference, not the code itself.

I'd check environment variables, config files, and secrets to ensure they match expected values in production.

Next, I'd verify external dependencies—like database endpoints, APIs, or service URLs—since local setups often use mocks or different endpoints.

I'd also check networking aspects such as DNS resolution, service discovery, or firewall rules that might differ in production.

Additionally, I'd look at resource constraints like memory limits or permissions, since containers may behave differently under restricted environments.

Based on findings, I'd align configuration and dependencies between environments rather than modifying the application code blindly.

10. You Fixed Issue But It Happened Again Next Day. Same production issue repeated.

What now?

Ans: If the same production issue happens again after I've already fixed it once, I'd treat it as a failure in root cause analysis rather than just reapplying the fix.

First, I'd stabilize the system again, but my main focus would be identifying the true root cause, not just the symptom I fixed earlier.

I'd review what was done in the previous fix—whether it was a temporary workaround or a permanent solution—and analyze logs, metrics, and system behavior over time to find patterns or triggers.

I'd also check for conditions under which the issue reoccurs, such as traffic patterns, scheduled jobs, memory leaks, or configuration resets.

Once identified, I'd implement a permanent fix and add preventive measures like alerts, monitoring, or safeguards to detect or avoid recurrence.

Finally, I'd document the incident and update runbooks so the team can respond faster if it happens again.

11. Traffic increased but pods are NOT scaling. What's wrong? Explain.

Ans: If HPA is not scaling despite increased traffic, I wouldn't assume scaling is broken immediately.

First, I'd verify what metric HPA is using—CPU, memory, or custom metrics—because traffic increase doesn't always reflect in CPU.

Then I'd check whether metrics are actually being collected correctly using Metrics Server or Prometheus. If metrics are missing or delayed, HPA won't react.

Next, I'd review HPA configuration—thresholds, min/max replicas—to ensure scaling is not capped or thresholds are too high.

I'd also check resource requests on pods, because incorrect requests can mislead HPA calculations.

Based on findings, I'd fix metrics pipeline or adjust scaling configuration rather than forcing manual scaling blindly.

12. Pods are restarting frequently, How would you handle it?

Ans: Frequent pod restarts usually indicate instability rather than a one-time failure.

I'd first check pod status and events using `kubectl describe pod` to see if it's `CrashLoopBackOff` or probe failures. Then I'd analyze container logs to identify whether the application is crashing or being killed.

I'd also verify liveness and readiness probes, since misconfigured probes can restart healthy containers.

Resource limits are another key factor—if memory limits are too low, the container may be getting `OOMKilled`.

Based on the root cause—application crash, probe issue, or resource constraint—I'd fix the configuration or code instead of repeatedly restarting pods.

13. Your application is suddenly hitting 403 errors. How do you investigate?

Ans: A sudden spike in 403 errors usually points to authorization or access control issues rather than application logic.

I'd first check recent changes in IAM policies, API gateway rules, ingress configurations, or WAF settings.

Then I'd verify whether authentication tokens or credentials have expired or changed.

I'd also check if there are IP restrictions or rate limiting rules blocking legitimate traffic.

Since this is a security layer issue, I'd focus on identifying what changed recently and restoring correct access rather than modifying application code.

14. What if DNS resolution is failing in the cluster?

Ans: If DNS resolution fails inside the cluster, I'd first verify CoreDNS pods are running and healthy, since they handle internal service discovery.

Then I'd test DNS resolution from inside a pod using tools like nslookup or dig to confirm whether it's a cluster-wide or pod-specific issue.

I'd also check the CoreDNS configuration and logs for errors, and verify that network policies or firewall rules are not blocking DNS traffic.

If needed, I'd restart CoreDNS or fix configuration issues, but only after identifying the root cause.

15. What is imagepullbackoff ? If it occurs how do you find the root cause?

Ans: ImagePullBackOff indicates the node is unable to pull the container image.

I'd first verify the image name and tag are correct, since typos are common.

Then I'd check if the image exists in the registry and whether it's accessible. For private registries, I'd validate imagePullSecrets and authentication.

I'd also check node-level network connectivity to ensure it can reach the registry.

Based on findings, I'd fix image references, credentials, or connectivity issues rather than retrying deployments blindly.

16. Your monitoring dashboard is showing low CPU but customers experiencing high latency in accessing the application. What would you do as a DevOps Engineer?

Ans: High latency with low CPU usage suggests the bottleneck is not compute but somewhere else in the request path.

I'd check database performance, including slow queries and connection limits, since DB latency often impacts response time.

I'd also analyze external API calls, network latency, or disk I/O performance.

Using tracing tools, I'd identify where time is being spent within the request lifecycle.

Based on findings, I'd optimize the slow component rather than scaling CPU, which would not solve the issue.

17. Partial failures occurred during the rolling update. What's your next move?

Ans: Partial failures during a rolling update usually indicate compatibility or readiness issues between versions.

I'd first check whether old and new versions are backward compatible, especially if there are API or schema changes.

Then I'd verify readiness probes to ensure traffic is only routed to fully initialized pods.

I'd also review rollout strategy settings like maxUnavailable and maxSurge.

If the issue persists, I'd pause the rollout or rollback to stabilize the system before fixing compatibility issues.

18. What if a node becomes notready?

Ans: If a node becomes NotReady, I'd first check node status and kubelet logs to understand the reason.

Common causes include resource exhaustion, network failure, or kubelet crash.

To protect the cluster, I'd cordon the node to stop scheduling and drain it if workloads are impacted.

Then I'd investigate the root cause—disk pressure, memory issues, or connectivity—and fix it before bringing the node back into service.

19. The application keeps failing, but logs don't show anything. What would you do?

Ans: If the application is failing but logs show nothing, I'd treat it as an observability gap rather than assuming no issue exists.

I'd rely on metrics, tracing, and health checks to understand system behavior.

I'd also verify whether logging is correctly configured and whether logs are being dropped or misrouted.

If needed, I'd increase log levels or add temporary debugging to capture more information.

Lack of logs itself is a signal that needs to be fixed alongside the main issue.

20. Your code works fine locally, but fails in the CI/CD pipeline. How do you handle this situation?

Ans: If code works locally but fails in the pipeline, I'd suspect differences in environment or execution context.

I'd compare runtime versions, dependencies, environment variables, and permissions between local and CI environments.

Then I'd analyze pipeline logs to identify the exact failing stage and error.

I'd try to reproduce the issue in a controlled environment similar to the pipeline.

Based on findings, I'd align environments or fix pipeline configuration rather than changing code blindly.